



Scallop Lending Audit

audited by Asymptotic

December 2, 2025

Summary

Asymptotic conducted a security audit of the Scallop Lending Protocol. The audit identified 22 issues across severity levels: 1 high, 6 medium, and 15 low. The high-severity finding was fully remediated, and the majority of medium and low findings were either remediated or acknowledged with justification. Additionally, 15 advisory-level suggestions were provided for code quality and best practices.

The Scallop team demonstrated strong engagement throughout the remediation process, addressing findings proactively with clear rationale for all design decisions.

Protocol Overview

Scallop is a lending protocol on Sui where users deposit collateral to borrow assets and earn/pay interest based on utilization. It supports multiple assets, including liquid staking tokens, and liquidates under-collateralized positions to stay solvent. Users must monitor their positions actively and keep healthy collateral ratios.

Trust Assumptions

Access Control: The system is non-custodial but has configurable access control. The admin can switch between three modes at any time: (1) allow all, (2) whitelist-only, or (3) reject all. All user operations check whitelist access. Users control their funds, while the admin controls all settings (risk parameters, listings, interest rates). A compromised or malicious admin could endanger user funds.

Oracles: The protocol relies on oracles (Pyth, Aftermath Finance/Haedal) for prices. If oracles are manipulated or stale, users may get liquidated unfairly. The protocol cannot prevent oracle-level attacks.

Obligation Locking: Users may lock their obligation to let another protocol manage it. While locked, user actions like borrowing, withdrawing, or repaying are restricted until unlocked. This enables automated strategies but requires trusting the external protocol to act correctly and release the lock. The authorized protocol can unlock anytime, though liquidation still only occurs if the position is actually unsafe.

Audited code

We audited the Move files in following directories at the given commit:

- `contracts` folder, **excluding**:
 - `test/` sub-folders
 - `test_coin/` folder
 - `vendors/` sub-folders
 - `*_test.move` files

Initial Commit: fc5a4a3

Final Commit: c24b851

Legend

Issue severity

- **Critical** — Vulnerabilities which allow account takeovers or stealing significant funds, along with being easy to exploit.
- **High** — Vulnerabilities which either can have significant impact but are hard to exploit, or are easy to exploit but have more limited impact.
- **Medium** — Moderate risks with notable but limited impact.
- **Low** — Minor issues with minimal security implications.
- **Advisory** — Informational findings for security/code improvements.

Legal Disclaimer

Terms and Conditions and Liability

This report is subject to the terms and conditions—including liability limitations—established between Asymptotic and the paying entity. By sharing code with Asymptotic, developers acknowledge and agree to these conditions. In the absence of executed agreements, liability is limited to the fees paid for these services.

Scope

This security audit report focuses specifically on reviewing the Move smart contract code. The audit does not analyze or make any claims about other components of the system, including but not limited to:

- Frontend applications and user interfaces
- Backend services and infrastructure
- Off-chain components and integrations
- Deployment procedures and operational security
- Third-party dependencies and external services

Limitations

The findings and recommendations in this report are limited to the Move code implementation and its immediate interactions within the Sui blockchain environment.

Creating proofs for specifications is on a “best effort” basis. We manually audited the code in instances where formal proofs were not technically feasible.

Issues

H-1: Unbounded Revenue Factor Enables Fund Drainage

Severity: **High**

Description

The `revenue_factor` parameter in interest model has no upper bound validation when configured by the `AdminCap` holder, allowing values exceeding 100%. This enables systematic theft of depositor funds through a sophisticated virtual accounting mechanism that gradually drains the shared cash pool without triggering aborts.

The `revenue_factor` is used to calculate `revenue_increased` based on `debt_increased`. This is purely an accounting operation, no actual funds are moved at this stage. As a result, values greater than 100% won't cause an immediate abort.

However, the protocol later attempts to extract real funds based on these virtual claims. If `balance_sheet.cash` contains sufficient funds, the protocol may claim revenue (partially from the users' deposit pool to cover above 100% revenue). This can lead to serious inconsistencies in the accounting and balances state, and ultimately cause aborts during legitimate user claims due to insufficient cash in the pool.

This is not just a parameter validation issue. It's a user fund safety vulnerability.

Recommendation

Assertions should be added to ensure that `revenue_factor` cannot be set higher than 100% (or preferably, a more conservative upper bound).

Remediation

✓ Remediated

✓ Commit: 9e8afc5

M-2: Market UID Access Through ext Function

Severity: **Medium**

Description

The ext function in app.move provides direct mutable access to the Market object's UID without any validation or constraints. While it requires AdminCap, this creates a dangerous bypass mechanism that allows AdminCap owner to manipulate critical protocol state outside of established safety controls.

The Market UID stores critical protocol configuration as dynamic fields, including the following:

- BorrowFeeVaultKey → Collected borrow fees storage
- WhitelistKey → Individual address permissions
- AllowAllKey → Global allow mode
- RejectAllKey → Global reject mode
- BorrowFeeKey → Borrow fee rates per asset
- BorrowFeeRecipientKey → Fee recipient address
- SupplyLimitKey → Maximum supply limits per asset
- BorrowLimitKey → Maximum borrow limits per asset
- IsolatedAssetKey → Asset isolation status

With mutable UID access, an attacker can perform any dynamic field operation (add, remove, modify) to manipulate protocol configuration. The most obvious attack vectors include enabling unlimited borrowing to drain protocol reserves, bypassing all whitelist restrictions, and removing or corrupting critical configuration fields, rendering the protocol inoperable. And these are just a few examples. Requiring an AdminCap reduces risk, but bypassing the protocol's safety mechanisms and restrictions still poses a serious threat.

Recommendation

Remove the ext function entirely as it represents a fundamental security anti-pattern.

If extensibility is truly required, all current uses of the ext function should be carefully reviewed and replaced with specific, validated administrative functions that preserve protocol safety guarantees.

At the very least, the ext function must include strict validation and constraints to allow only a narrowly defined set of safe actions.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: The ext function was replaced with abort 0, so the issue is fixed in the updated codebase.

However, previously deployed package versions remain fully callable on Sui, meaning an AdminCap holder could still invoke the old implementation through its original package ID.

M-6: Borrow Fee Recipient is Required but Ignored

Severity: **Medium**

Description

Borrow fee recipient configuration is non-functional, but mandatory for borrowing functionality.

The `borrow_internal` function fetches the configured `borrow_fee_recipient` address using `dynamic_field::borrow`. This field is added only via `update_borrow_fee_recipient` call.

But `borrow_internal` completely ignores recipient when distributing borrow fees. Instead of sending fees to the intended recipient, all fees are stored in the protocol's internal vault.

The protocol provides `update_borrow_fee_recipient` function for admins to configure fee recipients, and it must be called to add required configuration dynamic field. But this configuration has no effect on actual fee distribution.

While not a direct security vulnerability, this represents a functional defect that breaks an intended protocol feature and could lead to incorrect financial assumptions.

Recommendation

Implement actual fee transfer to recipient or remove unused recipient system entirely.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: Fee recipient is removed

M-4: Division by Zero Risk in Liquidation Amount Calculation

Severity: **Medium**

Description

The `max_liquidation_amounts` function contains a division by zero vulnerability in the denominator calculation at line 88. The function calculates:

$$\text{denominator} = \text{borrow_weight} \times (1 - \text{liq_penalty}) - \text{liq_factor}$$

If the expression $\text{borrow_weight} \times (1 - \text{liq_penalty})$ is less than or equal to `liq_factor`, the denominator becomes zero or negative, causing the function to abort.

This scenario is possible with the current validation limits for administrative configuration, where `liq_penalty` 20%, `liq_factor` 95%, and `borrow_weight` is unconstrained. When interest models and risk models are configured independently without cross-validation, specific parameter combinations can result in zero or negative denominators, causing aborts.

This can completely disable liquidations for specific parameter combinations, leading to protocol insolvency and bad debt accumulation.

Recommendation

Implement parameter validation to prevent dangerous combinations that can disable liquidations and threaten protocol solvency.

Add cross-parameter validation in all interest and risk models configuration functions to ensure the liquidation denominator always maintains at least some minimum safety margin above zero (e.g. 5%).

Remediation

Acknowledged

M-5: Interest Rate Kink Points Lack Bounds Validation

Severity: **Medium**

Description

The interest rate model uses kink points (`mid_kink`, `high_kink`) to determine utilization rate thresholds for different interest rate segments. Current validation during administrative configuration is incomplete, missing bounds checks that can cause division by zero errors and complete interest calculation failure.

The `calc_interest` function aborts if

- `mid_kink = 0` → line 170
- `high_kink = mid_kink` → line 182
- `high_kink ≥ 100%` → line 193

This cause complete interest calculation failure, which prevents all lending operations (including repayment of borrowed funds) as they call `update_interest_rates` → `calc_interest`.

The function also lacks upper bound validation for the rate parameters at kink points (`base_rate_per_sec`, `borrow_rate_on_mid_kink`, `borrow_rate_on_high_kink`, and `max_borrow_rate`). While their relative ordering is validated, extreme absolute values could cause economic disruption.

Large rate values combined with time calculations in borrow index updates may cause overflow conditions, particularly in the formula `old_borrow_index * interest_rate * time_delta` during interest accrual.

Recommendation

Add comprehensive validation in `create_interest_model_change`:

- `high_kink < scale`
- `mid_kink < high_kink`
- `mid_kink > 0`

Implement reasonable upper bounds for rate parameters like `max_borrow_rate ≤ MAX_REASONABLE_RATE` to prevent economically extreme configurations alongside the existing kink point validation.

Additionally, enforce that `util_rate ≤ 1` in the `calc_interest` function to maintain correct interest calculation.

Remediation

✓ Remediated

✓ Commit: 38256fa

M-3: Multiple Primary Rules Break Oracle System

Severity: **Medium**

Description

The oracle system's `determine_price` function strictly requires exactly one primary price feed but lacks validation to prevent administrators from configuring multiple primary rules for the same coin type.

`add_primary_price_update_rule_v2` allows unlimited rules. When two or more primary rules are accidentally configured, every price update transaction fails with `ONLY_SUPPORT_ONE_PRIMARY`, making the oracle completely unusable until admin intervention.

This creates a single point of failure where a simple configuration mistake can break the entire oracle system for affected assets.

Recommendation

Add primary rule limit validation to allow only one primary rule as it is expected by `determine_price`.

Use named constants for required primary feeds and secondary consensus threshold.

Remediation

✓ Remediated

✓ Commit: 92bf579

Notes: Code quality recommendation (named constants) not implemented but this is minor.

M-1: Pyth Oracle Fails with Positive Exponents

Severity: **Medium**

Description

The Pyth oracle integration contains a bug that causes transaction aborts when processing price feeds with positive exponents. The `get_pyth_price` function incorrectly assumes all Pyth exponents are negative, using `i64::get_magnitude_if_negative` which aborts if the exponent is positive. This makes entire asset categories unusable in the oracle system. Actual Pyth exponent semantics:

- Negative expo (-8): Price has 8 decimal places (common)
- Positive expo (+2): Price is scaled up by 10^2 (possible for low-value assets)

The `rule.move` decimal conversion logic (lines 42-47) assumes all exponents represent decimal places, incorrectly handling positive exponents during 9-decimal normalization.

Recommendation

Fix exponent handling in `pyth_adaptor` and decimal conversion in `rule`.

Additionally, add reasonable decimal limits (e.g. 18 instead of `U8_MAX` (255))

Remediation

Acknowledged

L-4: Borrow Weight Lacks Upper Limit

Severity: **Low**

Description

The `borrow_weight` parameter in interest models has no upper bound validation, allowing extreme values that make borrowing mathematically impossible or create unstable risk calculations. This parameter is used for risk-weighted debt calculations and affects both borrowing capacity and liquidation mathematics.

While there is no abort during configuration, extreme values like 100000% can make borrowing mathematically impossible for specific assets.

Recommendation

Add reasonable upper bound validation in `create__interest__model__change`.

Remediation

✓ Remediated

✓ Commit: 38256fa

L-9: Math Library `mul_div` Implementation Can Be Optimized

Severity: **Low**

Description

The `mul_div` implementations in `u64`, `u128`, and `u256` math modules contain inefficiencies and a potential division by zero vulnerability. These optimizations can improve performance and eliminate unnecessary complexity while fixing a safety issue.

`u64::mul_div` currently delegates to `u128::mul_div` with type casting. For `u64` values, direct multiplication to `u128` cannot overflow, making the complex overflow handling unnecessary.

`u128::mul_div` also can cast to `u256` avoiding overflow. But instead it uses `checked_mul` and `is_safe_mul` helpers, adding unnecessary function call overhead and code complexity.

`is_safe_mul` function in `u128` and `u256` causes division by zero when $x = 0$, even though mathematically $0 * y = 0$ never overflows.

Recommendation

For `u64::mul_div`: Replace with direct multiplication using `u128` casting. Since `u64 * u64` always fits in `u128`, no overflow checking needed. Just assert `c != 0` and result fits in `u64`.

For `u128::mul_div`: Use `u256` approach similar to `u64`. Cast to `u256`, multiply, divide, check result fits in `u128`.

Add early return for `x == 0 || y == 0` cases in `is_safe_mul` for `u256` before division check to eliminate division by zero. Remove unused versions of `checked_mul` and `is_safe_mul` functions.

Remediation

Acknowledged

L-1: Legacy V1 Rules Completely Ignored and Misleading

Severity: **Low**

Description

The oracle system implements two parallel rule management systems (V1 and V2) that appear to work together but V1 rules are completely ignored during validation, creating a silent security bypass where administrators can configure rules that will never be enforced. The legacy V1 system provides `add_rule` and `remove_rule` functions to manage `policy.rules`. However, the V2 system has transitioned to using dynamic fields for configuration.

The function `get_price_update_policy`, which is used during `confirm_request` validation, only reads from the V2 dynamic field and completely ignores legacy V1 `policy.rules`.

The same applies to the wrapper functions `add_primary_price_update_rule`, `remove_primary_price_update_rule`, `add_secondary_price_update_rule`, and `remove_secondary_price_update_rule`, which are all built on top of the deprecated legacy functions `add_rule` and `remove_rule`.

This results in a misleading and broken admin interface. Administrators may believe they are configuring rules via `add_rule` / `remove_rule`, but these changes have no effect on actual behavior.

Recommendation

Replace all legacy V1 functions with aborts to prevent any misleading or non-functional usage.

Integrate dynamic field initialization directly into the new function. This eliminates the need for calling `init_rules_df_if_not_exist()` separately, ensuring atomic and complete initialization of objects.

In long term remove V1 system and `init_rules_df_if_not_exist` function entirely.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: V1 functions properly abort preventing silent bypasses, but `init_rules_df_if_not_exist` still requires separate call and legacy rules field remains in struct due to upgrade limitations.

L-6: Multiple AuthorizedWitnessList Creation Enables Authorization Bypass

Severity: **Low**

Description

The borrow referral system contains two identical functions (`init` and `create_witness_list`, callable via public function `create_referral_witness_list`) that create `AuthorizedWitnessList` objects, violating the singleton principle.

The `create_borrow_referral` accepts any `AuthorizedWitnessList` object as parameter. An attacker with admin access could create multiple witness lists, self-authorize their witness in a rogue list, and bypass the intended authorization system when creating `BorrowReferral` objects.

Referral system appears unused in main codebase, reducing immediate impact

Recommendation

Remove duplicate `create_witness_list` function or make it `[test_only]` if it is intended for tests.

Alternatively, enforce singleton usage by specifying the exact active list to be used by the protocol, and adding runtime checks to ensure the provided list matches the expected one.

Remediation

Acknowledged

L-7: Incomplete Dynamic Field Cleanup in `destroy_borrow_referral`

Severity: **Low**

Description

The `destroy_borrow_referral` function only removes the `ReferralFeeKey` dynamic field but fails to clean up the `BorrowedKey` dynamic field and any custom configuration fields added via `BorrowReferralCfgKey`. This creates a resource leak where dynamic field data becomes permanently inaccessible after the parent object is deleted.

Recommendation

Implement complete cleanup of all associated dynamic fields.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: The `destroy_borrow_referral` does not remove any custom `BorrowReferralCfgKey<Cfg>` dynamic fields added via `add_referral_cfg`. Since dynamic fields cannot be enumerated without knowing their key types (and `BorrowReferralCfgKey<Cfg>` is generic), there is no reliable way to automatically clean them up.

Currently, `remove_referral_cfg` must be called manually before `destroy_borrow_referral` to clear all such fields, but the contract cannot enforce this sequence.

This is acceptable as a short-term remediation, but in the long term a different storage pattern should be used to avoid this limitation.

L-5: Missing Version Validation in Administrative Functions

Severity: **Low**

Description

All administrative functions in the app module that require `AdminCap` do not validate the protocol version before executing. While user-facing functions consistently call `version::assert_current_version` to ensure compatibility, administrative functions bypass this validation entirely.

Potentially, Admin operations on outdated protocol versions could corrupt market state.

Recommendation

Ensure administrative operations maintain the same version safety as user operations, preventing state corruption during protocol upgrades:

- Modify all admin functions to accept `version: &Version` parameter
- Add validation `version::assert_current_version(version);` at the beginning of each admin function.

Remediation

Acknowledged

L-3: Missing Parameter Validation in Limiter Configuration

Severity: **Low**

Description

The `add_limiter`, `create_limiter_params_change`, and `create_limiter_limit_change` functions lack critical input validation, allowing invalid configurations that can cause division by zero errors, infinite loops, or non-functional rate limiting.

Key validation gaps are:

- `build_segments` performs `outflow_cycle_duration / outflow_segment_duration` without checking if `outflow_segment_duration == 0`
- No validation that `outflow_limit`, `outflow_cycle_duration` and `outflow_segment_duration` are non-zero
- Missing check that `outflow_cycle_duration >= outflow_segment_duration`
- No validation that `outflow_cycle_duration % outflow_segment_duration == 0` for proper segment alignment

Division by zero causes immediate transaction failure during limiter creation. Zero segment duration causes division by zero in `add_outflow` and `count_current_outflow`. Overall, invalid limiters can prevent normal withdraw/borrow operations.

Recommendation

Add comprehensive parameter validation to all limiter functions:

- `outflow_limit > 0`
- `outflow_cycle_duration > 0`
- `outflow_segment_duration > 0`
- `outflow_cycle_duration % outflow_segment_duration == 0`

Consider renaming `add_limiter` to `create_limiter` since `wit_table::add` prevents duplicates, making it a creation operation rather than addition.

Remediation

✓ Remediated

✓ Commit: c24b851

L-2: Inconsistent Delay Validation

Severity: **Low**

Description

The admin delay extension functions have inconsistent validation, allowing malicious or compromised admins to permanently freeze protocol governance. While `extend_interest_model_change_delay` correctly limits increases to 1 epoch per call, `extend_risk_model_change_delay` and `extend_limiter_change_delay` have no validation whatsoever, allowing infinite delay values.

Malicious admin can set `risk_model_change_delay = u64::MAX`, making future risk model changes impossible to execute.

The current assertion `extend_interest_model_change_delay` restricts the parameter to `<= 1`, which means only 1 is effectively allowed. As a result, the parameter and check can be considered redundant, since the value is always incremented by 1 regardless.

And even the protected function allows unlimited cumulative delays over time through repeated calls.

Recommendation

Add the same validation that exists for interest model delays to both risk model and limiter delay functions. This ensures all delay extensions follow the same controlled incremental approach.

Consider removing the delay parameter and its associated assertion, and simply increment the value by 1 instead.

Implement reasonable upper bounds for total accumulated delays (e.g. 100 limit) to prevent indefinite governance lockout while maintaining security through time delays.

If infinite delays are intentionally allowed for protocol parameters freezing, create separate dedicated functions with clear naming like `"freeze_something"` rather than using extension functions, and add comprehensive documentation explaining the intended behavior.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: Fixed by disabling all delay extension functions. All three functions now validate `delay==0` and enforce `REASONABLE_MAX_DELAYS=0` cap. This prevents malicious admins from setting infinite delays or freezing protocol governance.

L-10: Missing Flash Loan Fee Limit

Severity: **Low**

Description

The `set_flash_loan_fee` function lacks upper bound validation, allowing admins to set extreme fees that effectively disable flash loans while still appearing functional. Unlike `update_borrow_fee` which properly validates `fee_numerator <= fee_denominator`, flash loan fees have no bounds checking despite documentation indicating a 0 - 10000 range.

Recommendation

Apply consistent validation to `set_flash_loan_fee` to enforce the documented 0-10000 (0-100%) range.

Consider reducing the maximum limits for borrow and/or loan fees.

Remediation

✓ Remediated

✓ Commit: 38256fa

L-8: Force Unlock Function Bypasses Version Control

Severity: **Low**

Description

The `force_unlock_unhealthy` function lacks the version control validation that is present in all other user functions.

Recommendation

Insert `version::assert_current_version(version)` at the beginning of the function to ensure it only operates on the correct protocol version. This requires `version: &Version` as the parameter in the function signature.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: There is a new `force_unlock` function, which properly validates version before execution. Old `force_unlock_unhealthy` deprecated with abort 0, simplifying from health-check-only to general whitelisted package unlock.

Since `force_unlock_unhealthy` originally lacked version validation, old package versions deployed on Sui can still call it. Protocol upgrade won't prevent old package invocations.

L-11: Decimal Operations Lack Overflow Validation Causing Delayed Abort

Severity: **Low**

Description

The decimal module performs high-precision calculations on **u64** values by converting them to **u256** with 18-decimal precision. However, arithmetic operations (**add**, **mul**, **div**, **pow**) lack overflow validation, allowing calculations to succeed even when results exceed the valid **u64** range.

The overflow is only detected later during **floor** conversion back to **u64**, causing transaction abort with unclear error context (no info which specific calculation caused the failure).

The existing saturating_**floor** function silently caps results to **u64** max value, creating incorrect financial calculations without error indication. This would corrupt protocol accounting and create unfair user outcomes.

Recommendation

Add assertions to ensure arithmetic operations results remain within valid **u64** range:

```
result.value <= (u64::max_value as u256) * WAD
```

Consider adding **mul_div** function for **Decimal** if needed (the protocol currently uses obsolete **FixedPoint32** for complex financial calculations, not **Decimal**).

Additionally improve **pow** function by pre-calculating **pow(from(2), 32)** as constant.

Remediation

Acknowledged

✓ **Commit:** 38256fa

Notes: Only minor optimization is added

L-12: APM Configuration and Coverage Gaps

Severity: **Low**

Description

The `set_apm_threshold` function accepts any `u8` value (0-255%) without validation. Setting threshold to 0% would trigger APM on any price increase, effectively blocking all borrowing operations.

Additionally, APM checks only occur during borrow operations via `check_is_collateral_price_fluctuate`, leaving withdrawal and other price-sensitive operations unprotected against manipulation.

For instance, `max_withdraw_amount` also depends on `available_borrow_amount_in_usd` (if there is any debt), which is price-sensitive. So if the price is manipulated, users could potentially withdraw more than they deposited

Recommendation

Add validation for the APM threshold to enforce a minimum allowable limit.

Consider calling APM checks during withdrawal operations.

Remediation

✓ **Remediated**

✓ **Commit:** c24b851

Notes: The current cap for the APM threshold is set to 10,000%, which appears excessively high.

L-13: APM Data Initialization and Staleness

Severity: **Low**

Description

APM protection is disabled for new assets and after periods of inactivity.

When assets are first listed, APM initializes with empty price history (price: 0, last_update: 0). The first borrowing operation can use any manipulated price since there's no historical data for comparison.

After 24+ hours without borrowing activity, all price history becomes stale. The `is_price_fluctuate` function finds no valid data and returns false, disabling APM protection.

Both scenarios create predictable vulnerability windows where attackers can manipulate prices without APM interference.

Recommendation

Initialize APM history with the current price instead of zeros. Allow the admin to provide the current price when calling `set_apm_threshold` or introduce a dedicated admin function to populate the APM price history using the current, legitimate price from the oracle (only if history is empty or stale). For newly listed assets, borrowing should only be enabled after the admin initializes the price history to ensure proper APM protection is in place.

Additionally, allow the admin to refresh the APM price history during quiet periods. This helps maintain protection by preventing asset history from becoming stale.

Remediation

✓ Remediated

✓ Commit: 38256fa

L-14: Precision Loss and Configuration Vulnerabilities in afSUI and haSUI

Severity: **Low**

Description

The `get_price` function in custom oracle rules for afSUI and haSUI applies `decimal::floor()` immediately after calculating the final price, then performs integer arithmetic for decimal formatting. This results in underpricing when `price_value_with_formatted_decimals` is calculated by multiplying with the decimals.

Admin functions lack input validation:

- `update_exchange_rate_constraint` accepts any min/max values without checking if min max or enforcing economic reality (staked assets should be 100% of base asset value)
- `update_oracle_config` accepts confidence tolerance of 0%, which would reject all Pyth prices since confidence is never exactly zero

Overall, the afSUI and haSUI implementations use identical `oracle_config` and `pyth_adaptor` logic, with only some differences in their rule logic.

Recommendation

Perform all price decimal formatting using decimal arithmetic instead of integer operations and apply floor only as the final step.

In `update_exchange_rate_constraint` validate `min_exchange_rate_bps <= max_exchange_rate_bps` and enforce minimum bounds 100%.

Add lower bound validation for confidence tolerance (>0) in `update_oracle_config`.

Consolidate shared logic into common libraries.

Remediation

✓ Remediated

✓ Commit: 38256fa

L-15: Flash Loan Consumes Entire User Payment Without Refunding Excess

Severity: **Low**

Description

The `repay_flash_loan` function in reserve module takes the entire user coin payment and treats any overpayment as protocol revenue, unlike other repayment functions that return excess funds to users. When users provide more tokens than required (`loan_amount + fee`), the excess is permanently kept by the protocol instead of being refunded (with no way to recover funds for user).

Recommendation

Modify `repay_flash_loan` to split the exact required amount and return excess to the user, consistent with the `repay` function pattern.

Alternatively, enforce exact payment by requiring `repaid_amount == loan_amount + fee` to prevent overpayment entirely.

Remediation

✓ Remediated

✓ Commit: 38256fa

A-11: Error Code Duplication

Severity: **Advisory**

Description

The x and math libraries use error codes starting from 0 which can conflict with other modules.

The error module contains duplicate flash loan error codes: `flash_loan_repay_not_enough_error` and `flash_loan_not_paid_enough` (never used)

There is also unused `DIVIDE_BY_ZERO` error in math module.

Recommendation

Remove unused error codes and update x library error codes to start from a unique non-zero value like it is done for `EInvalidPublisher` instead of using values from 0 to 3.

Remediation

✓ Remediated

✓ Commit: 38256fa

A-1: Validation Gaps in `confirm_request`

Severity: [Advisory](#)

Description

The oracle system's `confirm_request` function lacks proper validation for empty rule configurations and empty price feed vectors. When no rules are configured for a coin type, the system allows empty price update requests to pass validation, only to crash later in `determine_price` with confusing error messages.

The `confirm_request` function only validates that the number of receipts matches the number of configured rules, but doesn't validate:

- Minimum price feed requirements
- Empty rule configuration scenarios
- Primary feed existence

Recommendation

Add comprehensive validation to `confirm_request` with specific error codes indicate exact problems. This will ensure all invalid requests fail at validation time, not during price determination.

Remediation

Acknowledged

A-8: Price Feed Matching Overflow Risk and Hard-coded Parameters

Severity: [Advisory](#)

Description

The `price_feed_match` function contains a potential overflow vulnerability and uses hard-coded values for price difference calculations. While overflow is unlikely with current price ranges, the multiplication `value1 * scale` could theoretically overflow u64 limits with extreme price values.

Additionally, the tolerance parameters are hardcoded, preventing runtime configuration adjustments:

- `scale = 1000` (precision multiplier)
- `reasonable_diff_percent = 1` (1% tolerance)
- `reasonable_diff = 10` (calculated: $1 * 1000 / 100$)

Recommendation

Replace `diff` calculation with overflow-safe `mul_div` function and add named constants for tolerance parameters.

Consider administrative configuration functions for tolerance.

Remediation

Acknowledged

A-3: Code Duplication Across Oracle Modules

Severity: **Advisory**

Description

The oracle system contains extensive code duplication across multiple rule modules and registry systems. Nearly identical logic is repeated across Pyth, Supra, and Switchboard implementations, creating maintenance burdens, inconsistent behavior, and potential security vulnerabilities when fixes are applied to some modules but not others. Some of duplication areas are:

- pyth_rule functions `set_price_as_primary` and `set_price_as_secondary` are 99% identical (only differ in final call)
- Identical decimal conversion logic across Pyth, Supra, and Switchboard
- `switchboard_registry` in `switchboard_rule` and `switchboard_on_demand_rule` are 100% identical.

Recommendation

Extract common logic in helper functions/module. For example, the `switchboard_on_demand_rule` already demonstrates the correct pattern with its `get_switchboard_price` helper function that eliminates primary/secondary duplication.

Remediation

Acknowledged

A-12: Unused EMA Prices Field in XOracle

Severity: [Advisory](#)

Description

The XOracle struct contains an `ema_prices` field that is completely unused throughout the entire oracle system.

EMA prices are common in oracle systems for reducing volatility impact, and this field suggests EMA functionality that doesn't exist.

Recommendation

Remove the unused field unless EMA functionality is planned for immediate implementation. The current oracle system works without it.

Or document the intended purpose of this field.

Remediation

Acknowledged

A-5: Deprecated Standard Library APIs Usage

Severity: [Advisory](#)

Description

The codebase extensively uses deprecated Sui standard library APIs, which introduces potential risks for future compatibility and maintainability.

For example:

- The `fixed_point32_empower` module relies on the deprecated `std::fixed_point32` module and re-implements some missing functionality. However, the entire module is marked as deprecated.
- The deprecated `sui::math` module is still being used in the current math libraries.
- The codebase defines custom constants such as `U64_MAX`, `U128_MAX`, and `U256_MAX` instead of using the built-in functions `u64::max_value`, `u128::max_value`, and `u256::max_value`.

Newer APIs often include additional safety checks and optimizations. Accumulation of outdated patterns reduces code quality and developer productivity.

Recommendation

Replace `std::fixed_point32` usage with `std::uq32_32`, which provides all required functions with better performance and additional safety checks. Consider migrating `math::fixed_point32_empower` to use a newer type, or replacing it entirely with `std::uq32_32`.

Replace `sui::math` functions (e.g. `pow`, `min`) with individual type modules `u64`, `u128` and `u256`.

Replace custom constant definitions with built-in equivalents.

Also consider migration to `edition = "2024.beta"`. Move 2024 offers performance improvements, enhanced safety features, modern language syntax, and better tooling support that could reduce gas costs and improve code maintainability.

Remediation

Acknowledged

A-4: Public Entry Functions Create Dual Restrictions on Upgradability and Composability

Severity: [Advisory](#)

Description

The protocol contains a lot of public entry functions that impose both upgradability and composability restrictions simultaneously. The public entry pattern combines the worst limitations of both modifiers: permanent API lock-in from public and parameter/return type constraints from entry:

- public makes function signatures immutable across upgrades, preventing security enhancements or parameter additions
- entry restricts input arguments and requires return types to have drop ability, limiting protocol integration

Recommendation

Split public entry functions by purpose and:

- Use entry only for upgradable transaction endpoints that don't need external module access
- Use public only for composable APIs that other protocols can integrate with
- Avoid public entry combination unless both restrictions are genuinely required

Remediation

Acknowledged

A-2: Deprecated Reward System

Severity: **Advisory**

Description

The `set_incentive_reward_factor` function accepts parameters and performs storage operations for a reward system that is explicitly marked as deprecated and unused. The core reward calculation code is commented out with “@deprecated: this feature is no longer used”, yet the configuration function remains active, misleading administrators into believing they’re configuring functional rewards.

Overall, the entire supporting infrastructure remains active, including reward factor configuration, access key management, and user redemption functions that always return zero rewards.

Recommendation

Replace `set_incentive_reward_factor` function body with abort and a clear error message indicating the reward system is deprecated.

Consider also making `add_reward_key`, `remove_reward_key`, and `redeem_rewards_point` functions deprecated with abort statements.

In the long-term, schedule complete removal of the reward subsystem in the next major protocol upgrade.

Remediation

✓ Remediated

✓ Commit: 38256fa

Notes: `set_incentive_reward_factor` now uses abort 0. However, supporting infrastructure remains active: `add_reward_key`, `remove_reward_key`, and `redeem_rewards_point` still functional. These functions should also be replaced with abort 0 in long-term to prevent misleading administrators and users.

A-13: Hardcoded Configuration Values

Severity: **Advisory**

Description

The codebase contains multiple hardcoded configuration values embedded directly in functions instead of using named constants. These magic numbers reduce code maintainability, make configuration changes difficult, and create inconsistencies across similar functionality. Some examples are:

- Pyth uses hardcoded 30 seconds, while Switchboard and Supra use 60 seconds for price staleness validation
- Oracle modules use hardcoded values like 10 seconds for future price tolerance
- 100%-112% exchange rate defaults for afSUI and haSUI custom oracle rules and `ha-sui_exchange_rate_scale = 1_000_000`
- Hardcoded 10000 flash loan scale factor in reserve calculations
- `math::pow(10, 9)` hardcoded in `borrow_dynamics` and obligation for initial borrow index values
- decimal uses hardcoded 100 and 10_000 for percent and bps conversions
- Hardcoded APM values like 3600 and 24 are used in multiple places
- Seconds in year calculation inside `get_current_borrow_apr`

Recommendation

Replace hardcoded values with descriptive constants at module level.

Remediation

Acknowledged - Some important constants were added (flash loan scale, seconds in year, WAD), but many magic numbers remain throughout oracle rules, APM logic, and initialization values

A-6: Missing Interest Accrual Validation in Market Repay Function

Severity: [Advisory](#)

Description

The `market::handle_repay` function relies on `accrue_all_interests` being called immediately before it but lacks validation to ensure this critical prerequisite is met.

The function contains explicit comments and a TODO indicating this dependency, yet no runtime check exists to prevent incorrect usage that could lead to inaccurate debt calculations.

Recommendation

Modify `handle_repay` to include a timestamp parameter and validate that interest has been accrued for the asset type by checking `last_updated_by_type == now` at the function start.

Remediation

✓ Remediated

✓ Commit: `e2831a6`

A-10: Code Duplication in Obligation Creation

Severity: [Advisory](#)

Description

The `open_obligation_entry` function duplicates core logic of `open_obligation`. Both functions perform identical version checks, obligation creation, and event emission, creating maintenance overhead.

Recommendation

Create a private `create_obligation_internal` function containing the shared logic (version check, obligation creation, event emission) and have both public functions call it with different post-processing (immediate sharing vs hot potato return).

Remediation

✓ Remediated

✓ Commit: 36adb31

A-9: Potential Gas DoS Risk from Unbounded Asset Loops

Severity: [Advisory](#)

Description

Multiple critical protocol functions iterate through all protocol assets or user position assets, creating potential DoS vectors if asset counts grow significantly. Global market functions like `accrue_all_interests` loop through all protocol assets (20 currently), while per-obligation functions loop through user assets (typically 1-5). If either count grows substantially, gas consumption could exceed transaction limits and disable affected operations.

Global asset loops in `accrue_all_interests` and `update_interest_rates` could disable the entire protocol if gas limits are exceeded, while per-obligation loops in value calculation functions could lock out users with many positions.

Current risk is manageable but requires proactive monitoring as the protocol grows to prevent future DoS scenarios.

Recommendation

Evaluate processable limits for asset counts and enforce them when adding new assets to the system or a user's obligation.

If the protocol scales beyond these limits in the future, consider implementing more advanced and optimized solutions, including grouping, pagination, or batching.

For protocol assets, you may use the hot potato pattern to process all assets within a PTB across multiple transactions, if needed.

Only implement sophisticated approaches if they're actually needed, as they will add complexity to the system.

Remediation

Acknowledged

A-7: Flash Loan Fee Calculation Lacks Validation

Severity: [Advisory](#)

Description

The `borrow_flash_loan_internal` function misses validation for fee discount parameters that could cause underflow. While the function is currently called with hardcoded safe parameters (0, 1) providing no discount, the lack of validation creates risk for future modifications or integrations.

The base fee calculation `amount * fee_rate` could theoretically overflow, though this is extremely unlikely given realistic flash loan amounts and fee rates.

Recommendation

Add discount validation `fee_discount_numerator <= fee_discount_denominator` and `fee_discount_denominator > 0`.

Use `mul_div` for base fee calculation to prevent theoretical overflow.

Remediation

✓ Remediated

✓ Commit: 38256fa

A-14: Generic Error Codes Reduce Clarity

Severity: [Advisory](#)

Description

Multiple functions throughout the protocol use generic error codes or rely on runtime panics instead of providing specific, informative error messages. Sometimes relying on panic is acceptable when panic is impossible during normal execution. However, the protocol overuses panic, even in situations that are both possible and common. This makes debugging difficult and provides poor user experience when transactions fail.

Some examples are:

- Scale parameters validation relies on runtime panics during rational number construction rather than explicit validation
- Lack of amount validation and failures with unclear balance split errors (like in `borrow_flash_loan_i`)
- Dynamic field access without existence checks, or borrowing from `ac_table` and `wit_table`
- Relying on underflow during math operation instead of explicit checks
- Generic `invalid_params_error` used instead of specific validation errors

Recommendation

Implement comprehensive input validation with specific error codes.

Remediation

Acknowledged

A-15: Zero Amount Operations Execute Successfully Without Meaningful Effect

Severity: [Advisory](#)

Description

Several user-facing functions accept zero amounts or perform operations that have no effect, wasting gas and potentially confusing users about transaction outcomes. Functions execute successfully but perform no meaningful state changes.

Some examples are:

- `deposit_collateral` accepts zero-value coins and processes them without validation
- `mint` accepts zero deposit amounts and performs calculations unnecessarily
- `withdraw_collateral` accepts zero amounts (though limited by max calculation)
- `switch_to_whitelist_mode/add_whitelist_address/remove_whitelist_address` doesn't check current state, potentially doing nothing

Recommendation

Add comprehensive input validation and state checks to prevent no-op operations.

Remediation

✓ Remediated

✓ Commit: `e890c56`

Notes: `remove_whitelist_address` and `switch_to_whitelist_mode` still could waste gas on no-op operations but won't cause functional issues

About the Auditor

Asymptotic provides a white-glove, formal-verification-based auditing service for Sui smart contracts.

Asymptotic Team

Lead Auditor

Mykhailo Burakhin — Principal Auditor

- Conducted detailed security audits and gas optimizations for complex DeFi applications
- Developed and audited ERC-20, ERC-721, and ERC-1155 token contracts, including staking, vesting, auctions, and launchpads
- Engineered cross-chain bridges using ERC-4337 account abstraction, multi-signature wallets, and upgradeable proxy contracts
- Implemented protections against anti-sniping and front-running, and secure onboarding with MPC and zero-knowledge proofs
- Over 15 years of experience architecting secure, high-performance systems with a strong focus on cryptography, blockchain security, and advanced C++ engineering

Auditor

Danylo Provilskyi — Blockchain Engineer

- Built solo an EVM protocol achieving \$10M+ TVL with formal verification
- Shipped 6+ production Web3 projects from architecture to deployment, leading 15+ developers
- Selected for Uniswap Hook Incubator for DeFi protocol innovation
- Conducted security audits on EVM and Algorand using Echidna, Slither, Certora, and Foundry
- Won ETHWarsaw 2025 Spring Hackathon with cross-chain AI Agent on ViaLabs and OP Stack